

# Animación de llamas en OLED 128x32 (Arduino)

Transcripción de la conversación con Claude

Nota: para no repetir cientos de líneas de bytes idénticas, los bloques de bitmaps de los 28 frames de la animación se muestran una sola vez completos (en el archivo animacionLlamas.h final) y en el resto de la conversación se abrevian como [datos de bitmap omitidos, ver archivo adjunto].

---

## Usuario

hola, cómo puedo hacer que una serie de frames se reproduzcan en una pantalla de 128x32 OLED (0.91"), para hacer una animación?

ejemplo: [adjuntó un archivo .h con 2 frames de bitmap para SSD1306, 128x32, 1 bit/pixel]

## Asistente

¡Hola! Para animar una serie de frames en una OLED 128x32 (típicamente con el controlador SSD1306), la idea general es:

1. Tener cada frame como un arreglo de bytes independiente (con nombres distintos).
2. Guardarlos en un arreglo de punteros a esos arreglos.
3. En el loop(), dibujar el frame actual con drawBitmap(), esperar un tiempo, y pasar al siguiente.

Paso 1: nombrar cada frame distinto (el ejemplo del usuario repetía el mismo nombre de variable y el mismo include guard en los dos frames, lo cual da error de compilación si se incluyen juntos).

```
// frame0.h
#ifndef FRAME0_BITMAP_H
#define FRAME0_BITMAP_H
#include <Arduino.h>

static const unsigned char PROGMEM frame0_bitmap[512] = {
    0x00,0x00, /* ... resto de los 512 bytes ... */
};
#endif

// frame1.h
#ifndef FRAME1_BITMAP_H
#define FRAME1_BITMAP_H
#include <Arduino.h>

static const unsigned char PROGMEM frame1_bitmap[512] = {
    0x00,0x00, /* ... resto de los 512 bytes ... */
};
#endif
```

Paso 2: arreglo de frames + loop de animación, usando Adafruit\_SSD1306:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "frame0.h"
#include "frame1.h"

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 32
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

const unsigned char* frames[] = {
```

```

    frame0_bitmap,
    frame1_bitmap
};

const int numFrames = sizeof(frames) / sizeof(frames[0]);
const int frameDelay = 100; // ms entre frames

void setup() {
    if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
        for (;;)
    }
    display.clearDisplay();
}

void loop() {
    for (int i = 0; i < numFrames; i++) {
        display.clearDisplay();
        display.drawBitmap(0, 0, frames[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        delay(frameDelay);
    }
}

```

Puntos clave: drawBitmap(x, y, bitmap, w, h, color) dibuja el bitmap completo; clearDisplay() antes de cada frame evita mezclar píxeles; delay(frameDelay) controla el FPS; cada frame de 512 bytes se guarda en flash gracias a PROGMEM.

Alternativa sin bloqueo, usando millis() en vez de delay(), para no congelar el resto del programa (lectura de sensores, botones, etc.):

```

unsigned long lastFrameTime = 0;
int currentFrame = 0;

void loop() {
    // ... otras tareas del programa ...

    if (millis() - lastFrameTime >= frameDelay) {
        lastFrameTime = millis();
        display.clearDisplay();
        display.drawBitmap(0, 0, frames[currentFrame], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        currentFrame = (currentFrame + 1) % numFrames;
    }
}

```

Pregunta de cierre: ¿usás Adafruit\_SSD1306 o U8g2? ¿Cuántos frames pensás usar en total?

## Usuario

y cómo lo podría añadir aquí para que suceda entre las líneas del poema que dice "Para poder amar.", y "Me consumo en mi fuego,"?

## Asistente

El asistente no tenía a la vista el sketch del poema y pidió que lo compartiera, junto con cómo se mostraba el texto, si ya existía una estructura de temporización, y si el poema y la animación compartirían pantalla al mismo tiempo o en una pausa entre ambas líneas.

## Usuario

Adjuntó el sketch completo "Queja" de Alfonsina Storni: pantalla OLED I2C 128x32, Adafruit\_GFX + Adafruit\_SSD1306, poema controlado por un potenciómetro B10K en el pin A0 (cada posición del

potenciómetro corresponde a un verso), con palabras clave mostradas más grandes ("efecto grito"), texto centrado horizontal y verticalmente, interlineado configurable, para Arduino Uno R4 WiFi. 13 versos en total, funciones mostrarVerso() y limpiarPalabra().

## Asistente

Como el poema avanza con el potenciómetro (no con tiempo), la solución es: cuando se detecta el paso del verso 8 ("Para poder amar.") al verso 9 ("Me consumo en mi fuego,"), disparar la animación completa (bloqueante, con delay()) y después mostrar el verso 9 normalmente.

1. Frames en archivos separados, corrigiendo el problema de nombres/include guards repetidos (frame0.h y frame1.h).

2. Includes en el sketch principal:

```
#include "frame0.h"
#include "frame1.h"
```

3. Arreglo de frames de la animación:

```
// Frames de la animación que se muestra entre "Para poder amar." y "Me consumo en mi fuego,"
const unsigned char* framesAnimacion[] = {
    frame0_bitmap,
    frame1_bitmap
};
const int numFramesAnimacion = sizeof(framesAnimacion) / sizeof(framesAnimacion[0]);
const int frameDelayAnimacion = 100; // ms entre frames, ajustá a gusto
```

4. Función que reproduce la animación:

```
void reproducirAnimacion() {
    for (int i = 0; i < numFramesAnimacion; i++) {
        display.clearDisplay();
        display.drawBitmap(0, 0, framesAnimacion[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        delay(frameDelayAnimacion);
    }
}
```

5. Modificación del loop() para detectar la transición del verso 8 al 9:

```
void loop() {

    int valorPot = analogRead(POT_PIN);
    int indiceActual = map(valorPot, 0, 1023, 0, totalVersos - 1);

    if (indiceActual != indiceAnterior) {

        if (indiceAnterior == 8 && indiceActual == 9) {
            reproducirAnimacion();
        }

        mostrarVerso(versosPantalla[indiceActual], palabraGrande[indiceActual]);
        Serial.println(misVersos[indiceActual]);
        indiceAnterior = indiceActual;
    }

    delay(20);
}
```

Nota importante: como el potenciómetro puede "saltar" varios índices de golpe, la condición == solo dispara si se pasa exactamente por esa transición. Alternativa más robusta ante saltos:

```
if (indiceAnterior < 9 && indiceActual >= 9) {
    reproducirAnimacion();
}
```

Pregunta de cierre: ¿la animación debería dispararse solo la primera vez que se cruza esa transición, para no repetirla si el usuario mueve el potenciómetro adelante y atrás varias veces por esa zona?

## Usuario

Adjuntó un nuevo archivo con 28 frames (frame 0 a frame 27) de la animación de llamas/fuego para la pantalla 128x32, en formato PROGMEM de 512 bytes cada uno (método Ordered/Bayer). Cada frame usaba nombres de variable ya distintos (intentoAnimacion00\_bitmap a intentoAnimacion27\_bitmap), pero los 28 bloques repetían el mismo include guard INTENTOANIMACION01\_BITMAP\_H.

[Datos de bitmap de los 28 frames omitidos aquí — el contenido final y completo está en animacionLlamas.h, adjunto a esta conversación]

## Asistente

Confirmación de que "crear una new tab" en el IDE de Arduino simplemente crea un archivo aparte (en este caso .h) en la misma carpeta del sketch, que luego se incluye con #include "animacionLlamas.h" desde el .ino principal.

Problema detectado: los 28 frames compartían el mismo include guard (INTENTOANIMACION01\_BITMAP\_H) repetido 28 veces. Un include guard solo puede aparecer una vez por archivo — se necesita un único #ifndef/#define al principio y un único #endif al final, envolviendo los 28 arrays (que ya tenían nombres distintos: intentoAnimacion00\_bitmap a intentoAnimacion27\_bitmap).

Se generó el archivo animacionLlamas.h corregido y consolidado, con esta estructura:

```
// =====
// animacionLlamas.h
// Frames de la animación (efecto llamas / fuego) para OLED 128x32
// Formato: 128x32, 1 bit/pixel, LSB first
// =====

#ifndef ANIMACIONLLAMAS_H
#define ANIMACIONLLAMAS_H

#include <Arduino.h>

// ----- Frame 0 -----
static const unsigned char PROGMEM intentoAnimacion00_bitmap[512] = { ... };

// ----- Frame 1 -----
static const unsigned char PROGMEM intentoAnimacion01_bitmap[512] = { ... };

// ... (frames 2 a 26 con la misma estructura) ...

// ----- Frame 27 -----
static const unsigned char PROGMEM intentoAnimacion27_bitmap[512] = { ... };

// ----- Arreglo de punteros a todos los frames -----
const unsigned char* const framesLlamas[] = {
    intentoAnimacion00_bitmap, intentoAnimacion01_bitmap, intentoAnimacion02_bitmap,
    intentoAnimacion03_bitmap, intentoAnimacion04_bitmap, intentoAnimacion05_bitmap,
    intentoAnimacion06_bitmap, intentoAnimacion07_bitmap, intentoAnimacion08_bitmap,
    intentoAnimacion09_bitmap, intentoAnimacion10_bitmap, intentoAnimacion11_bitmap,
    intentoAnimacion12_bitmap, intentoAnimacion13_bitmap, intentoAnimacion14_bitmap,
    intentoAnimacion15_bitmap, intentoAnimacion16_bitmap, intentoAnimacion17_bitmap,
    intentoAnimacion18_bitmap, intentoAnimacion19_bitmap, intentoAnimacion20_bitmap,
    intentoAnimacion21_bitmap, intentoAnimacion22_bitmap, intentoAnimacion23_bitmap,
    intentoAnimacion24_bitmap, intentoAnimacion25_bitmap, intentoAnimacion26_bitmap,
    intentoAnimacion27_bitmap
};

const int numFramesLlamas = sizeof(framesLlamas) / sizeof(framesLlamas[0]);
```

```
// Delay entre frames (ms). Ajustalo para hacer la llama más lenta o más rápida.
const int frameDelayLlamas = 60;

#endif // ANIMACIONLLAMAS_H
```

Archivo entregado como descarga: animacionLlamas.h

## Usuario

ya corregí lo que me dijiste

## Asistente

Faltaba conectar animacionLlamas.h con el sketch principal. Pasos indicados:

1. Incluir el header nuevo arriba del .ino:

```
#include "animacionLlamas.h"
```

2. Agregar la función que reproduce la animación:

```
void reproducirAnimacion() {
  for (int i = 0; i < numFramesLlamas; i++) {
    display.clearDisplay();
    display.drawBitmap(0, 0, framesLlamas[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
    display.display();
    delay(frameDelayLlamas);
  }
}
```

3. Dispararla en el loop() al cruzar del verso 8 al 9:

```
if (indiceActual != indiceAnterior) {

  if (indiceAnterior < 9 && indiceActual >= 9) {
    reproducirAnimacion();
  }

  mostrarVerso(versosPantalla[indiceActual], palabraGrande[indiceActual]);
  Serial.println(misVersos[indiceActual]);
  indiceAnterior = indiceActual;
}
```

Se pidió confirmar cómo había quedado animacionLlamas.h antes de seguir con la integración.

## Usuario

Compartió la última versión del sketch principal poemaQueja.ino, con un pequeño ajuste propio: en versosPantalla[], el primer verso pasó de "Alfonsina Storni," a "Alfonsina Storni" (sin coma).

## Asistente

Se integró la animación de llamas al sketch principal y se entregaron dos archivos completos y listos para compilar juntos en la misma carpeta:

poemaQueja.ino — cambios respecto a la versión anterior del usuario:

```
#include "animacionLlamas.h" // Frames de la animación de fuego (pestaña nueva del sketch)

...

if (indiceActual != indiceAnterior) {

  // Si el usuario cruza del verso 8 ("Para poder amar.") hacia el 9
```

```

// ("Me consumo en mi fuego,") o mas alla, mostramos la animacion de llamas.
if (indiceAnterior < 9 && indiceActual >= 9) {
    reproducirAnimacion();
}

mostrarVerso(versosPantalla[indiceActual], palabraGrande[indiceActual]);
Serial.println(misVersos[indiceActual]);
indiceAnterior = indiceActual;
}

...

void reproducirAnimacion() {
    for (int i = 0; i < numFramesLlamas; i++) {
        display.clearDisplay();
        display.drawBitmap(0, 0, framesLlamas[i], SCREEN_WIDTH, SCREEN_HEIGHT, SSD1306_WHITE);
        display.display();
        delay(frameDelayLlamas);
    }
}

```

Todo lo demás del sketch (versos, mostrarVerso(), limpiarPalabra()) se mantuvo igual a la versión del usuario.

animacionLlamas.h — sin cambios respecto al archivo consolidado del turno anterior (28 frames + arreglo framesLlamas[] + numFramesLlamas + frameDelayLlamas).

Instrucciones finales: crear el sketch con ambos nombres exactos (poemaQueja.ino y animacionLlamas.h) como pestañas en la misma carpeta, y compilar. Con el potenciómetro moviéndose lento, la llama se reproduce completa (28 frames a 60 ms c/u, aproximadamente 1,7 segundos) antes de que aparezca el verso 9. El parámetro para ajustar la velocidad es frameDelayLlamas dentro de animacionLlamas.h.